

Synopsis Report of 5stage RV64 processor

1. Introduction

How does a processor execute billions of instructions every second?

At the heart of a processor is a simple idea: **do more work in parallel**. Instead of waiting for one instruction to completely finish before starting the next, a pipelined processor divides instruction execution into multiple stages and allows several instructions to be processed simultaneously.

In this project, I designed and implemented a **64-bit RV64 RISC-V processor using a fivestage pipeline**:

IF → ID → EX → MEM → WB

The goal of this project is not simply to build a processor that executes instructions, but to understand what happens **inside the machine**—how instructions move through the pipeline, how the datapath and control logic interact, and what happens when instructions depend on each other or a branch changes the flow of execution.

The processor is implemented in **SystemVerilog RTL** and verified through simulation. The design includes the program counter, instruction memory, decoder, register file, immediate generator, ALU, branch unit, data-memory interface, control logic, and pipeline registers.

This project is my step toward understanding processor microarchitecture from the **instruction level down to the RTL implementation**.

Table of Contents:

Chapter	Title	Main Questions I Answer
1	Motivation	What is a processor pipeline? Why is pipelining required? What is the objective of this project?
2	RISC-V ISA and Processor Fundamentals	What is RISC-V? Why did I choose RV64? What are the instruction formats, registers, and basic instruction types?
3	Five-Stage Pipeline Architecture	What are the limitations of a single-cycle processor? How does pipelining improve instruction throughput?
4	Five-Stage Pipeline Architecture	What are the IF, ID, EX, MEM, and WB stages? What operations are performed in each stage?
5	Processor Datapath	How does an instruction travel through the processor? How are the PC, register file, ALU, memories, and pipeline registers connected?
6	Instruction Fetch and Decode	How is an instruction fetched? How does the processor decode an instruction? How are registers and immediate values obtained?
7	Execute Stage and ALU	How are arithmetic, logical, comparison, and address-generation operations performed? How does the ALU control work?
8	Memory Access and Write-Back	How are load and store instructions handled? How is the result written back to the register file?
9	Pipeline Registers	Why are pipeline registers required? What information is transferred between IF/ID, ID/EX, EX/MEM, and MEM/WB stages?
10	Data Hazards and Hazard Handling	What are data hazards? What is a RAW dependency? How are stalls, forwarding, or other hazard mechanisms used?
11	Control Hazards and Branch Handling	What happens when a branch or jump is encountered? How are incorrect instructions flushed from the pipeline?
12	Processor Control Unit	How does the control unit generate control signals? How are control signals propagated through the pipeline?
14	Verification and Testing	How did I verify instruction execution, ALU operations, branches, loads, stores, and pipeline behavior?
16	Limitations and Future Improvements	What features are currently missing? What can be added in the future, such as caches, branch prediction, or better hazard handling?
17	Conclusion	What did I successfully design and implement? What did I learn from the project?
18	References	Which books, papers, specifications, lectures, and documentation were used?

Motivation

When I was in **12th grade while preparing for the JEE**, I learned about many interesting concepts such as how engines work, magnetism, optics, and electromagnetic induction. By that time, I had developed a fairly good understanding of how many technologies work. However, there was one major technology that I still did not understand: **how a computer actually works at the hardware level**.

I have always been curious about technology. In 2016, my father bought a smartphone that eventually stopped working. Around 2020, out of curiosity, I decided to open it and explore what was inside. At that time, I did not know much about electronics or semiconductor devices. I removed the motherboard and even tried to extract one of the ICs. After further experimentation, I discovered a small, shiny structure inside the package.

I was fascinated by how something so small could perform such a complex task. I started researching and learned that the structure was related to **silicon**, the fundamental semiconductor material used to build modern integrated circuits. What surprised me most was that an extremely small piece of silicon could contain millions or even billions of transistors and enable a smartphone to perform enormous amounts of computation.

This experience made me curious about what was actually happening **inside the silicon**. How could such a tiny piece of material perform calculations, store information, and execute programs? What happens inside a processor when I open an application or perform a simple operation?

To understand these questions, I decided to pursue **Electronics Engineering at Walchand College of Engineering, Sangli**, with a strong interest in semiconductor and digital hardware design. During my first year, I studied **digital system design**, and the more I learned, the more questions I had about what happens beyond logic gates.

I gradually became interested in **digital circuits, computer architecture, processors, memory systems, and VLSI design**. This led me from studying individual digital circuits to understanding how those circuits can be combined to build a complete processor.

The **five-stage pipelined RISC-V processor** developed in this project is a step toward understanding that complete picture. Instead of treating a processor as a black box, I want to understand how an instruction moves through the hardware, how the datapath and control unit work, how different instructions execute simultaneously through pipelining, and how the processor ultimately performs computation using digital circuits.

This project therefore represents one step in my journey from simply wondering “**What is inside a computer?**” to actually **designing and implementing the hardware that makes computation possible**.

But what is **computation**?

A few months ago, I went to the library to search for some books on **computer architecture**. Since my institute is quite old—it was established in **1945**—the library contains many old books.

While searching, I came across an old book about **vacuum tubes**. The book had a brown-colored cover and looked quite old.

I simply flipped through its pages for almost an hour. At first, I assumed that vacuum tubes were an old technology and that the book would be simple and easy to understand. However, after reading it, I realized that my assumption was completely wrong.

I found that vacuum-tube technology was far more complicated than I had expected. By the late 1940s, engineers had pushed vacuum tubes to their limits, and these books were trying to explain how to design some of the **most sophisticated vacuum tubes** and how they could be implemented for **computational purposes**.

At that time, **TI (Texas Instruments)** was one of the leading laboratories working on semiconductor technology. They developed an early **transistor** that consumed far less power and was much smaller and more reliable than vacuum tubes.

But the biggest innovation was the development of the **integrated circuit (IC)** by **Jack Kilby**. This invention revolutionized the world of computing and electronics.

Today, the computing power that once required an entire room filled with machines in the 1940s can fit inside my pocket. **Silicon has compressed a giant machine into something smaller than a grain of rice.**

Today, we are moving toward **quantum computing**, which is fundamentally different from classical computing and operates according to the principles of quantum mechanics. It has the potential to revolutionize computing once again in the future.

This raises an interesting question: **What remains the same through all these technological changes—from vacuum tubes to transistors to qubits?**

The answer is the **mathematics and theory behind computation**. A transistor is not computation itself; it is a very effective physical device for implementing computation. If, in the

future, we discover something far smaller than a 2 nm transistor that can exhibit similar computational behavior and can be reliably connected and arranged, then I would try to build a computational system using that new technology—not necessarily using transistors.

The physical implementation may change, but the fundamental principles of computation can remain the same.

Can we improve computational performance without increasing transistor count? In other words, can we achieve better performance with the same transistor budget? This is a question I have been exploring for a long time. My study focuses on how better architecture and microarchitecture can improve performance within a fixed hardware budget.

RISC-V ISA and Processor Fundamentals

WHY We need ISA

My first major source for learning digital systems was MIT OpenCourseWare. I watched around 20 lectures on digital system design, where I learned about two major approaches to instruction set architecture: **RISC (Reduced Instruction Set Computer)** and **CISC (Complex Instruction Set Computer)**.

The **ISA (Instruction Set Architecture)** defines the behavior and interface of a computer from the programmer's perspective. At the hardware level, a computer can be viewed as a finitestate machine, and computation is governed by a defined set of rules and constraints. We cannot perform arbitrary operations directly; the hardware provides a specific set of operations that the programmer can use.

This leads to an interesting question:

If an algorithm can be described mathematically or computationally, can we implement hardware to execute that algorithm?

My answer is **yes**.

Let us take a simple example: an algorithm for calculating the sum of the first **N natural numbers**.

To implement this algorithm directly in hardware, we need only a few basic components: a **counter, an adder, and registers**. The counter keeps track of the current value of N , while the adder accumulates the result. Registers, implemented using elements such as **D flip-flops**, store the intermediate values between clock cycles.

This type of specialized hardware implementation is common in ASIC design, particularly in domains such as **DSP (Digital Signal Processing)**, where algorithms can be mapped efficiently into dedicated hardware datapaths.

However, the problem becomes much more difficult when the algorithm involves **complex data structures, dynamic memory access, recursion, irregular control flow, or complicated software abstractions**. It is no longer practical to build a dedicated hardware circuit for every possible algorithm.

So, how do we solve this problem?

One solution is to build **programmable hardware**.

Instead of designing a completely new circuit for every algorithm, we design a general-purpose hardware architecture that can execute many different algorithms through **programs or instructions**. This is one of the fundamental ideas behind programmable processors.

In this way, we move from **fixed-function hardware** toward **general-purpose programmable computation**, allowing the same hardware to execute a wide variety of algorithms while changing only the instructions or program.

My first explanation introduced the concept of ISA, but I did not explain **why I chose RISC-V** for this project.

In the MIT lecture series, the professor mentioned that studying the **x86 ISA** in depth can take at least six months. I think that is reasonable, because x86 is an extremely complex ISA with many features accumulated over decades.

One important characteristic of x86 is its **variable-length instruction encoding**. An x86 instruction can range from **1 byte up to 15 bytes**. This provides benefits for code density and gives compilers and programmers considerable flexibility. However, this complexity also makes the hardware implementation much more difficult.

For example, instruction decoding becomes significantly more complicated because the processor must determine the length and boundaries of each instruction before it can properly decode and execute it. This complexity propagates into other parts of the processor, including **instruction fetch, decoding, branch prediction, speculation, and pipeline control**. Even implementing a relatively small number of instructions can become unnecessarily complicated when dealing with a CISC ISA such as x86.

The goal of my project is not to reproduce the complexity of a commercial x86 processor. Instead, I want to study and implement important **microarchitectural concepts**, such as:

- Pipeline architecture
- Branch prediction
- Data forwarding
- Hazard detection and handling

To focus on these concepts without making the ISA itself the main source of complexity, I chose **RISC-V**.

RISC-V follows the RISC philosophy of keeping the instruction set relatively simple and making the hardware easier to implement. Its base ISA uses **fixed-width 32-bit instructions**, with a straightforward encoding structure and relatively simple decoding compared with x86.

This simplicity allows me to focus on the **microarchitecture** rather than spending most of the project dealing with the complexity of instruction decoding.

Therefore, RISC-V is a good platform for this project: it provides a simple ISA while giving me enough freedom to explore how **pipelining, branch prediction, forwarding, and other microarchitectural techniques** can improve processor performance.

Since I am working on this project alone and have a limited amount of time, implementing the complete RISC-V ISA would be difficult. Therefore, I chose to implement a **subset of the RISC-V ISA**, including the fundamental instruction types along with the **M extension (integer multiplication and division)**.

This allows me to keep the processor manageable while still supporting the essential instructions needed to explore the microarchitecture.

RV64 Instruction Set — Decoding Reference

Base ISA (RV64I) — R / I / S / B / U / J types. Excludes M (multiply/divide) and C (compressed) extensions.

opcode | rd | funct3 | rs1 | rs2 | funct7 | immediate

[RV64] = RV64I-only instruction

R-Type — Register–Register (opcode 0110011 / 0111011 for *W variants)

Mnemonic	opcode	funct3	funct7	Bit layout [31:0]	Operation
ADD	0110011	000	0000000	0000000 rs2 rs1 000 rd 0110011	rd = rs1 + rs2

SUB	0110011	000	0100000	0100000 rs2 rs1 000 rd 0110011	rd = rs1 - rs2
AND	0110011	111	0000000	0000000 rs2 rs1 111 rd 0110011	rd = rs1 & rs2
OR	0110011	110	0000000	0000000 rs2 rs1 110 rd 0110011	rd = rs1 rs2
XOR	0110011	100	0000000	0000000 rs2 rs1 100 rd 0110011	rd = rs1 ^ rs2
SLL	0110011	001	0000000	0000000 rs2 rs1 001 rd 0110011	rd = rs1 << rs2[5:0]
SRL	0110011	101	0000000	0000000 rs2 rs1 101 rd 0110011	rd = rs1 >> rs2[5:0] (logical)
SRA	0110011	101	0100000	0100000 rs2 rs1 101 rd 0110011	rd = rs1 >> rs2[5:0] (arithmetic)
SLT	0110011	010	0000000	0000000 rs2 rs1 010 rd 0110011	rd = (rs1 < rs2) ? 1:0 (signed)
SLTU	0110011	011	0000000	0000000 rs2 rs1 011 rd 0110011	rd = (rs1 < rs2) ? 1:0 (unsigned)
ADDW	0111011	000	0000000	0000000 rs2 rs1 000 rd 0111011	rd = sext(rs1[31:0] + rs2[31:0]) [RV64]
SUBW	0111011	000	0100000	0100000 rs2 rs1 000 rd 0111011	rd = sext(rs1[31:0] - rs2[31:0]) [RV64]
SLLW	0111011	001	0000000	0000000 rs2 rs1 001 rd 0111011	rd = sext(rs1[31:0] << rs2[4:0]) [RV64]
SRLW	0111011	101	0000000	0000000 rs2 rs1 101 rd 0111011	rd = sext(rs1[31:0] >> rs2[4:0]) (logical) [RV64]
SRAW	0111011	101	0100000	0100000 rs2 rs1 101 rd 0111011	rd = sext(rs1[31:0] >> rs2[4:0]) (arith) [RV64]

I-Type — Immediate Arithmetic (opcode 0010011)

Mnemonic	opcode	funct3	funct7/imm	Bit layout [31:0]	Operation
ADDI	0010011	000	imm[11:0]	imm[11:0] rs1 000 rd 0010011	rd = rs1 + sext(imm)
ANDI	0010011	111	imm[11:0]	imm[11:0] rs1 111 rd 0010011	rd = rs1 & sext(imm)
ORI	0010011	110	imm[11:0]	imm[11:0] rs1 110 rd 0010011	rd = rs1 sext(imm)
Mnemonic	opcode	funct3	funct7/imm	Bit layout [31:0]	Operation
XORI	0010011	100	imm[11:0]	imm[11:0] rs1 100 rd 0010011	rd = rs1 ^ sext(imm)
SLTI	0010011	010	imm[11:0]	imm[11:0] rs1 010 rd 0010011	rd = (rs1 < sext(imm)) ? 1:0 (signed)
SLTIU	0010011	011	imm[11:0]	imm[11:0] rs1 011 rd 0010011	rd = (rs1 < sext(imm)) ? 1:0 (unsigned)

SLLI	0010011	001	000000\ shamt[5:0]	000000 shamt[5:0] rs1 001 rd 0010011	rd = rs1 << shamt[5:0]
SRLI	0010011	101	000000\ shamt[5:0]	000000 shamt[5:0] rs1 101 rd 0010011	rd = rs1 >> shamt[5:0] (logical)
SRAI	0010011	101	010000\ shamt[5:0]	010000 shamt[5:0] rs1 101 rd 0010011	rd = rs1 >> shamt[5:0] (arithmetic)

I-Type — Loads (opcode 0000011)

Mnemonic	opcode	funct3	imm	Bit layout [31:0]	Operation
LB	0000011	000	offset imm[11:0]	imm[11:0] rs1 000 rd 0000011	rd = sext(M[rs1+sext(imm)] [7:0]) rd = zext(M[rs1+sext(imm)] [7:0])
LBU	0000011	100	offset imm[11:0]	imm[11:0] rs1 100 rd 0000011	rd = sext(M[rs1+sext(imm)] [15:0])
LH	0000011	001	offset imm[11:0]	imm[11:0] rs1 001 rd 0000011	rd = zext(M[rs1+sext(imm)] [15:0])
LHU	0000011	101	offset imm[11:0]	imm[11:0] rs1 101 rd 0000011	rd = sext(M[rs1+sext(imm)] [31:0]) rd = zext(M[rs1+sext(imm)]
LW	0000011	010	offset imm[11:0]	imm[11:0] rs1 010 rd 0000011	[31:0]) [RV64] rd = M[rs1+sext(imm)] [63:0]
LWU	0000011	110	offset imm[11:0]	imm[11:0] rs1 110 rd 0000011	[RV64]
LD	0000011	011	offset imm[11:0]	imm[11:0] rs1 011 rd 0000011	

I-Type — Jump &

System (various opcodes)

Mnemonic	opcode	funct3	imm	Bit layout [31:0]	Operation
JALR	1100111	000	offset imm[11:0]	imm[11:0] rs1 000 rd 1100111	rd=PC+4 ; PC=(rs1+sext(imm))&~1
ECALL	1110011	000	000000000000	000000000000 00000 000 00000 1110011	Environment call (syscall)
EBREAK	1110011	000	000000000001	000000000001 00000 000 00000 1110011	Breakpoint trap

I-Type — RV64 Word Immediate (opcode 0011011)

Mnemonic	opcode	funct3	funct7/imm	Bit layout [31:0]	Operation
ADDIW	0011011	000	imm[11:0]	imm[11:0] rs1 000 rd 0011011	rd = sext(rs1[31:0]) + sext(imm)) [RV64]
SLLIW	0011011	001	0000000\shamt[4:0]	0000000 shamt[4:0] rs1 001 rd 0011011	rd = sext(rs1[31:0]) << shamt[4:0]) [RV64]
SRLIW	0011011	101	0000000\shamt[4:0]	0000000 shamt[4:0] rs1 101 rd 0011011	rd = sext(rs1[31:0]) >> shamt[4:0]) (logical) [RV64]
SRAIW	0011011	101	0100000\shamt[4:0]	0100000 shamt[4:0] rs1 101 rd 0011011	rd = sext(rs1[31:0]) >> shamt[4:0]) (arith) [RV64]

S-Type — Stores (opcode 0100011)

Mnemonic	opcode	funct3	imm	Bit layout [31:0]	Operation
SB	0100011	000	imm[11:5] imm[4:0]	imm[11:5] rs2 rs1 f3 imm[4:0] opcode	M[rs1+sext(imm)] = rs2[7:0]
SH	0100011	001	imm[11:5] imm[4:0]	imm[11:5] rs2 rs1 f3 imm[4:0] opcode	M[rs1+sext(imm)] = rs2[15:0]
SW	0100011	010	imm[11:5] imm[4:0]	imm[11:5] rs2 rs1 f3 imm[4:0] opcode	M[rs1+sext(imm)] = rs2[31:0]
SD	0100011	011	imm[11:5] imm[4:0]	imm[11:5] rs2 rs1 f3 imm[4:0] opcode	M[rs1+sext(imm)] = rs2[63:0] [RV64]

B-Type — Branches (opcode 1100011) — imm encodes signed byte offset, always ×2

Mnemonic	opcode	funct3	imm	Bit layout [31:0]	Operation
BEQ	1100011	000	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1==rs2: PC += sext(imm<<1)
BNE	1100011	001	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1!=rs2: PC += sext(imm<<1)
BLT	1100011	100	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1<rs2 (signed): PC += sext(imm<<1)
BLTU	1100011	110	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1<rs2 (unsigned): PC += sext(imm<<1)
BGE	1100011	101	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1>=rs2 (signed): PC += sext(imm<<1)
BGEU	1100011	111	imm[12 10:5 4:1 11]	im[12\ 10:5] rs2 rs1 f3 im[4:1\ 11] opcode	if rs1>=rs2 (unsigned): PC += sext(imm<<1)

U-Type — Upper Immediate

Mnemonic	opcode	imm	Bit layout [31:0]	Operation
LUI	0110111	imm[31:12]	imm[31:12] rd opcode	rd = imm << 12 (loads upper 20 bits)
AUIPC	0010111	imm[31:12]	imm[31:12] rd opcode	rd = PC + (imm << 12)

J-Type — Unconditional Jump

Mnemonic	opcode	imm	Bit layout [31:0]	Operation
JAL	1101111	imm[20 10:1 11 19:12]	imm[20\ 10:1\ 11\ 19:12] rd opcode	rd = PC+4 ; PC += sext(imm<<1) [20-bit signed offset, ×2]

RV64M — Multiply / Divide Extension Decoding Reference

All 14 instructions are pure R-type. *w variants are RV64-only (32-bit operands, sign-extended 64-bit result).

opcode[6:0] | rd[11:7] | funct3[14:12] | rs1[19:15] | rs2[24:20] | funct7[31:25] = 0000001

[RV64] = RV64-only (*W variants)

RV64M — Multiply (opcode 0110011, funct7 = 0000001) — operates on full 64-bit registers

Mnemonic	opcode	funct3	funct7	Bit layout [31:0]	Operation	Notes
MUL	0110011	000	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = (rs1 \times rs2)[63:0]$	Lower 64 bits of product
MULH	0110011	001	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = (rs1 \times s rs2)[127:64]$	Upper 64 bits — signed × signed
MULHU	0110011	011	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = (rs1 \times u rs2)[127:64]$	Upper 64 bits — unsigned × unsigned
MULHSU	0110011	010	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = (rs1 \times su rs2)[127:64]$	Upper 64 bits — signed × unsigned

RV64M — Divide & Remainder (opcode 0110011, funct7 = 0000001) — 64-bit signed/unsigned

Mnemonic	opcode	funct3	funct7	Bit layout [31:0]	Operation	Notes
DIV	0110011	100	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = rs1 \text{ div}(s) rs2$	Signed div; rounds toward zero
DIVU	0110011	101	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = rs1 \text{ div}(u) rs2$	Unsigned division
REM	0110011	110	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = rs1 \text{ rem}(s) rs2$	Signed remainder; sign = dividend
REMU	0110011	111	0000001	0000001 rs2 rs1 f3 rd 0110011	$rd = rs1 \text{ rem}(u) rs2$	Unsigned remainder

RV64M — 32-bit Word Variants (opcode 0111011, funct7 = 0000001) — result signextended to 64 bits

Mnemonic	opcode	funct3	funct7	Bit layout [31:0]	Operation	Notes
MULW	0111011	000	0000001	0000001 rs2 rs1 f3 rd 0111011	$rd = \text{sext}((rs1 \times rs2)[31:0])$	Lower 32-bit product, sext to 64
DIVW	0111011	100	0000001	0000001 rs2 rs1 f3 rd 0111011	$rd = \text{sext}(rs1[31:0] \text{ div}(s) rs2[31:0])$	Signed 32-bit div, sext result

DIVUW	0111011	101	0000001	0000001 rs2 rs1 f3 rd 0111011	rd = sext(rs1[31:0] div(u) rs2[31:0])	Unsigned 32-bit div, sext result
REMW	0111011	110	0000001	0000001 rs2 rs1 f3 rd 0111011	rd = sext(rs1[31:0] rem(s) rs2[31:0])	Signed 32-bit remainder, sext
REMUW	0111011	111	0000001	0000001 rs2 rs1 f3 rd 0111011	rd = sext(rs1[31:0] rem(u) rs2[31:0])	Unsigned 32bit remainder, sext

Decoding Key

$\text{fun}^{\text{c}_t}_7 = 0000001$ on any R-type instruction signals the M extension. Base-ISA R-type uses 0000000 or 0100000 — so a single $\text{fun}^{\text{c}_t}_7$ bit [25] set to 1 identifies all M instructions. **Edge-**

Case Semantics

Condition	DIV / DIVW	DIVU / DIVUW	REM / REMW	REMU / REMUW
Divide by zero	returns -1 (all ones)	returns $2^L - 1$ (all ones)	returns dividend	returns dividend
Signed overflow ($\text{MIN_INT} \div -1$)	returns MIN_INT	—	returns 0	—

above all are supported instruction for my core

Why Do I Need a Pipelined Processor?

When I see a pipeline, I think of the assembly line at Tata Motors.

Since the early days of computer architecture, after the 8051 series, the computer industry has tried to make processors faster. For a long time, clock frequency was a key performance metric. Everyone wanted their processor to run at the highest possible frequency.

Pipelining is like breaking a long task into smaller tasks.

For example, if fetching and decoding an instruction in the same clock cycle takes too much time, I can insert a register between these two operations and perform them in separate stages. This reduces the amount of work that has to be completed in a single clock cycle, allowing the processor to run at a higher clock frequency.

But does pipelining only increase the clock frequency?

My answer is no.

I also designed a non-blocking cache, where I used an FSM with two states: `TAG_FETCH` and `TAG_COMPARE`. These two states perform different tasks. One state fetches the tag from the tag memory, while the other compares the fetched tag with the requested tag.

If both operations are performed sequentially in one clock cycle, the clock period has to be long enough to accommodate both operations:

TAG_FETCH + TAG_COMPARE

However, if I divide them into two stages, each stage only needs to perform one operation. If both operations have approximately the same delay, the critical path can theoretically be reduced by about half, allowing the clock frequency to approach twice the original frequency.

For example, if `TAG_FETCH` and `TAG_COMPARE` together require a 1 GHz clock period, dividing them into two balanced stages could theoretically allow a clock frequency of around 2 GHz.

But is the throughput actually increased?

Not necessarily.

Imagine there is only one car in an assembly line. We divide the manufacturing process into multiple stages, but there is still only one car going through the line. The first car enters, goes through all the stages, and eventually finishes. Then the next car starts.

This is where the most important concept of pipelining comes in.

The goal of pipelining is not simply to increase the clock frequency. The important goal is to increase throughput by allowing multiple operations to be processed simultaneously.

For example, suppose instruction execution has two stages:

Stage 1 → Fetch

Stage 2 → Decode

Without pipelining, we might do:

Instruction 1: Fetch → Decode

Instruction 2: Fetch → Decode

But with pipelining, we can overlap the operations:

Cycle 1: Instruction 1 → Fetch

Cycle 2: Instruction 1 → Decode + Instruction 2 → Fetch

Cycle 3: Instruction 2 → Decode + Instruction 3 → Fetch

Now multiple instructions are inside the pipeline at the same time.

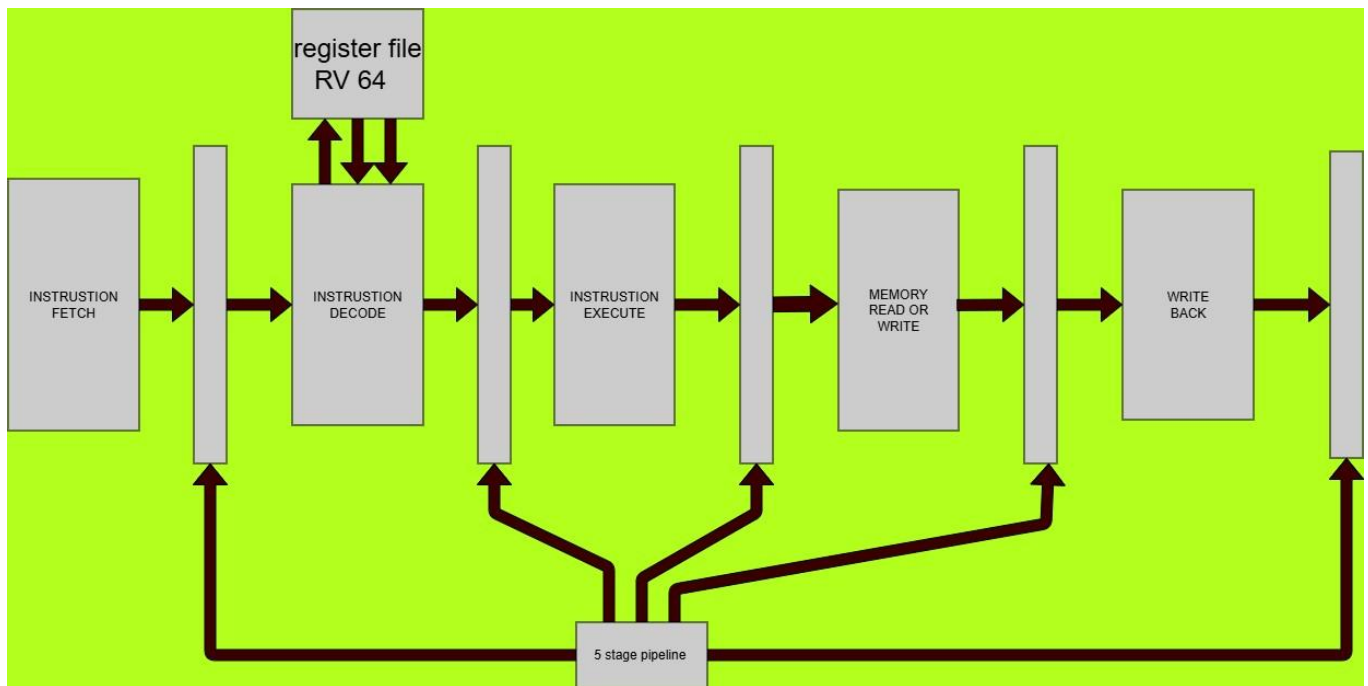
This is similar to an assembly line: while one car is being painted, another car can be assembled, and another can be inspected.

So, pipelining can increase clock frequency because it divides a long combinational path into shorter stages. But the real performance benefit comes from overlapping multiple operations, which increases throughput.

That is why I would say:

Pipelining is not about making one operation finish faster. It is about allowing more operations to be completed per unit of time.

Five-Stage Pipeline Architecture



above image shows the classical 5-stage pipeline processor.

Why 5 stages? Why not more? Why not fewer?

When I was explaining the importance of pipelining, I used the example of `TAG_FETCH` and `TAG_COMPARE`, which are two different operations.

Suppose `TAG_COMPARE` takes t time to compare the tag, and `TAG_FETCH` takes t time to fetch the tag from memory.

If we perform both operations in the same clock cycle, the total time required is:

$$t + t = 2t$$

Therefore, the maximum clock frequency is:

$$f = 1 / 2t$$

If we divide these operations into two pipeline stages, each stage takes t time. Therefore, the clock period becomes t , and the frequency becomes: $f = 1 / t$

So, ideally, the frequency doubles, and because the two operations can be overlapped for different requests, the **throughput also doubles**.

But what happens when the two stages do not have the same delay?

Suppose TAG_COMPARE takes $2t/3$ time, while TAG_FETCH takes $4t/3$ time.

If we perform both operations in the same clock cycle, the total delay is:

$$2t/3 + 4t/3 = 2t$$

Therefore, the clock frequency

is: $f = 1 / 2t$

Now, suppose we pipeline these two operations.

The clock period of the pipeline must be at least as long as the **slowest stage**. In this case,

TAG_FETCH takes $4t/3$, so the new clock period becomes:

$$T_{clk} = 4t/3$$

Therefore, the new frequency

is: $f = 3 / 4t$

So the frequency increases from:

$$1 / 2t \rightarrow 3 / 4t$$

The frequency improvement is only **1.5×**, not **2×**.

For example, if the original processor runs at **1 GHz**, after pipelining it would ideally run at around **1.5 GHz**, rather than 2 GHz.

And this is one of the important problems in pipeline design:

The pipeline clock is determined by the slowest pipeline stage.

Therefore, simply dividing a processor into more stages does not guarantee a proportional increase in frequency.

This also leads to an important question:

If adding more pipeline stages can increase frequency, why don't we simply divide the processor into 20, 30, or 50 stages?

The answer involves **stage imbalance, pipeline-register overhead, hazards, branch penalties, power, and increasing latency**. That is one of the reasons the classical processor uses a practical number of stages rather than endlessly increasing the pipeline depth.

Making a combinational circuit faster is relatively easy: you can simply divide the combinational logic into smaller sections, and you will generally get a predictable improvement.

However, designing a pipeline for a **sequential machine** is much more difficult. That is why I decided to drop the pipelining idea for my non-blocking cache processor.

The output of sequential logic depends not only on the current input but also on the **current state of the machine**. When we pipeline a sequential machine, we introduce additional dependencies between pipeline stages.

To handle these dependencies, we need additional logic to resolve or bypass them. This increases the **area footprint** and makes the design more complicated.

It also makes **verification and debugging much more difficult**, because the behavior of the state machine is now spread across multiple pipeline stages.

So, while pipelining a combinational datapath can be relatively straightforward, **pipelining a sequential state machine is a much more complex design problem**.

In a 5-stage pipeline architecture, we divide the processor into five stages: **Instruction Fetch, Instruction Decode, Execute, Memory Access, and Writeback**. These are well-known stages, so I do not need to do extensive research to determine their approximate functionality, area, or timing.

However, to design a **real pipeline**, we need to understand the timing reports. By studying the timing of the circuit, we can determine where to insert pipeline registers and how to divide the logic into different stages.

First, we specify the target clock frequency of the design. Then, we divide the combinational logic in such a way that the delay of each pipeline stage is as balanced as possible.

The goal is to make sure that **no single stage becomes the critical path** and that each stage can complete its operation within the target clock period.

Processor Datapath

Data Path and Control Path in Microarchitecture

When I studied the **8051 in my 4th semester**, we performed practicals involving assembly language programming for the 8051 processor.

In the 8051, there is an **Accumulator (A)**, and there is a general rule for many operations: the accumulator must be used as either an operand or the destination.

I noticed that whenever I wanted to transfer or process information, I often had to go through the accumulator. I would first move the data into the accumulator, perform the operation, store the result there, and then use another instruction to transfer the information from the accumulator to the final destination.

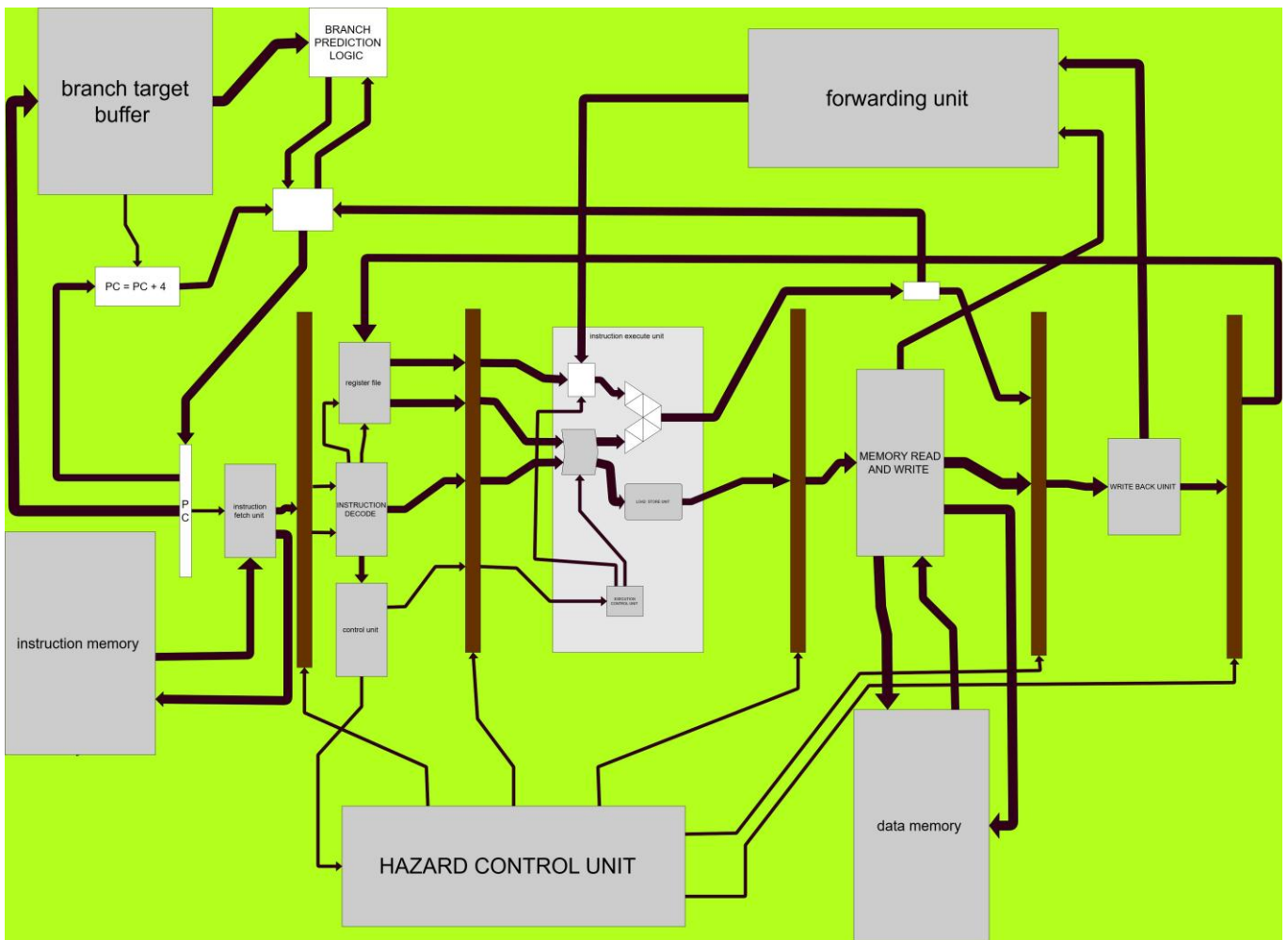
This creates a particular **data-flow path**: a significant amount of data has to pass through the accumulator.

I think of it like **two rivers connected by a canal**. The rivers may be large and capable of carrying a lot of water, but the amount of water that can flow between them is limited by the small canal connecting them.

Similarly, even if the rest of the processor has sufficient resources, forcing data to repeatedly pass through a single accumulator can become a bottleneck.

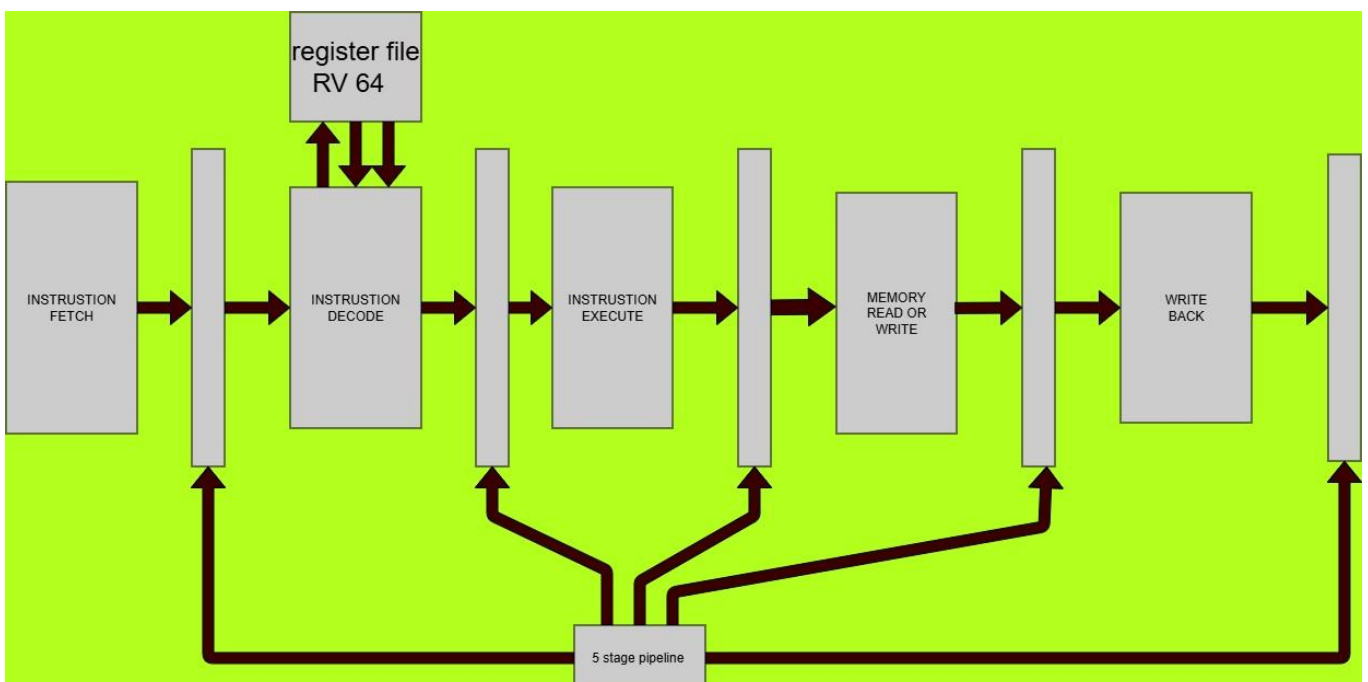
This is one of the limitations I noticed in the 8051 architecture.

In our design, we don't want to impose this kind of unnecessary restriction on the data path. We want the datapath to provide more flexible routes for moving data between registers, functional units, and memory, while the **control path decides which route the data should take.**



In the above design, the **smaller arrows represent the control path**, while the **larger arrows represent the data path**.

In the data memory, there are two separate paths: one for **read operations** and another for **write operations**. Both paths support the transfer of a full **8-byte word**. in above design how data path is there is represented by thicker arrow



I read in a book that **making common things faster, more efficient, and better** is one of the fundamental principles of computer architecture, and I am applying the same principle to my hardware design.

A pipeline is not necessarily just a set of registers. **Whatever hardware is divided into multiple stages so that different operations can be performed concurrently can be considered pipelined.** That is the principle I am applying to my processor.

My target clock frequency is **1 GHz**, which means one clock cycle is **1 ns (1000 ps)**. Therefore, the combinational delay of each pipeline stage should ideally be less than about **900 ps**, leaving some timing margin and some time for the pipeline registers to capture and transfer the data.

The image above shows the datapath of my processor. The pipeline is implemented directly in the hardware datapath shown in the diagram.

But what about the **memory and register file**?

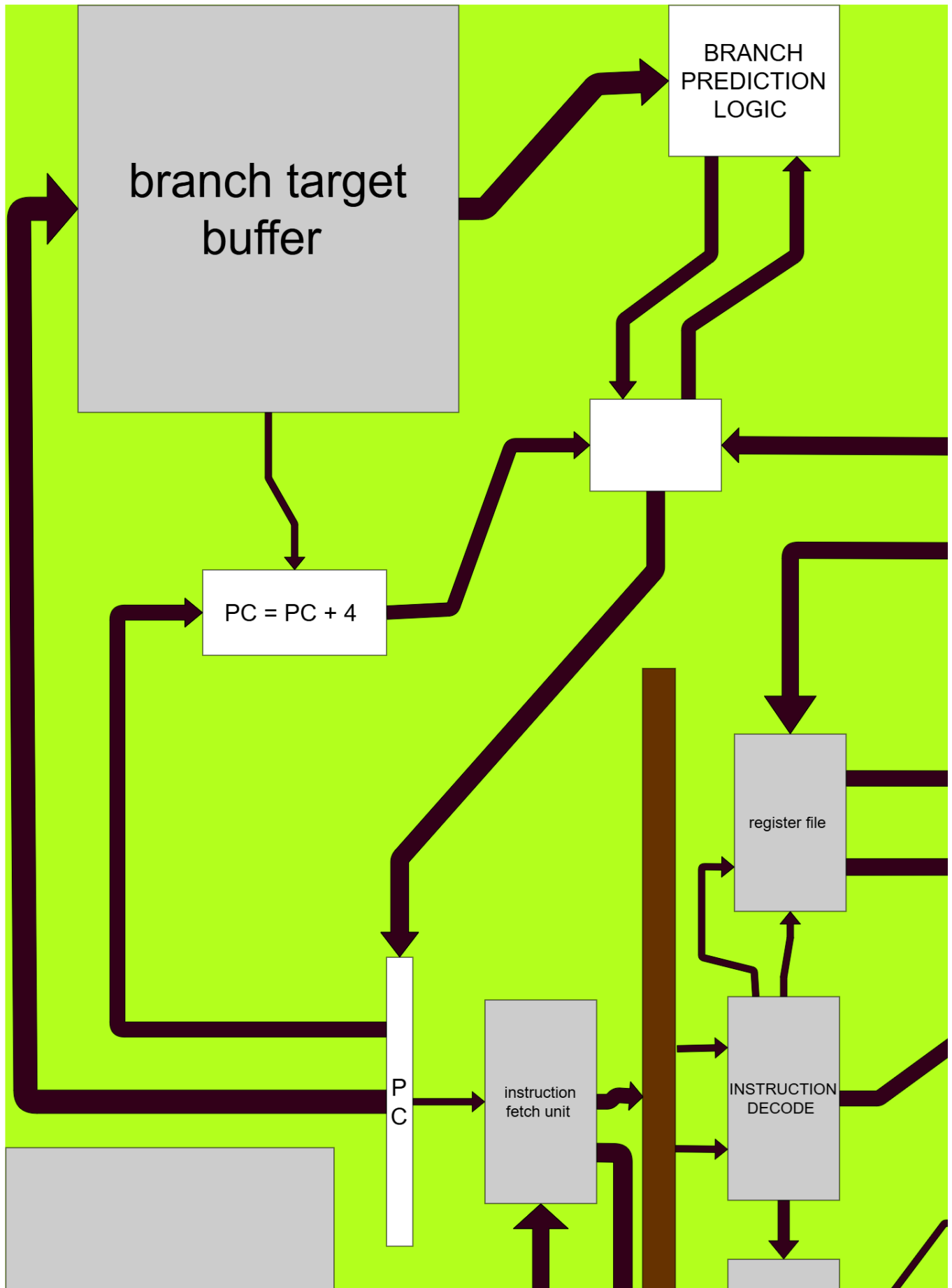
There is a final stage that looks somewhat like a register in the diagram. Data flows from the **write-back stage to the register file**. I forgot to draw the register file explicitly in the diagram. In my design, when data arrives at the register file during the current cycle, it becomes available as an output in the **next cycle**. Therefore, can we also consider the register file operation as a **pipelined operation**?

-> **NO**

My target is to divide the total delay into **1000 ps (1 ns) clock cycles**. In the worst-case scenario, the operation takes **2 cycles** to complete, so I will get the data after 2 cycles.

This is also pipelining because, during those two cycles, **I can send another request to the hardware instead of waiting for the first request to complete.**

Instruction Fetch and Decode



As I said, **common things must be made faster and more efficient**.

Most modern processors use **out-of-order execution**, so we can tolerate some data latency. If the required data is not immediately available, the processor can still continue executing other independent instructions to some extent.

But what happens if the **instruction itself is not available**? Can the core execute future instructions, or will the processor stall?

From what I have learned, there is no way for the processor to execute an instruction if it cannot fetch the required instruction. A more advanced speculative machine can tolerate this to a very limited extent, but fundamentally, the processor eventually needs the instruction stream to continue execution. So, what is the solution?

Can I say that **my fetch unit needs to be much faster**?

I am also working on a project involving a **non-blocking cache**, where I am designing an L2 cache. In that design, I give higher priority to the **L1 instruction cache (I-cache)**. The idea is to make the path for instruction fetching as fast as possible.

I do this because I realized what could happen if the fetch unit requests an instruction and a **TLB miss** occurs, and the page-table entry indicates that the page is stored on an SSD. Now we have a serious problem: the instruction fetch can experience an extremely large latency.

To handle such situations, we need a mechanism to stop or stall the processor when necessary.

Practically, this requires **backpressure**. Here, I want to share an experience I had while designing a **matrix multiplication unit** that takes three matrices as inputs and produces an output:

$$Y = A \times B + C$$

After I completed the design, I encountered a problem during verification. I fed the MAC unit with data in a streaming manner and waited for the output. After a certain number of cycles, I received the expected output.

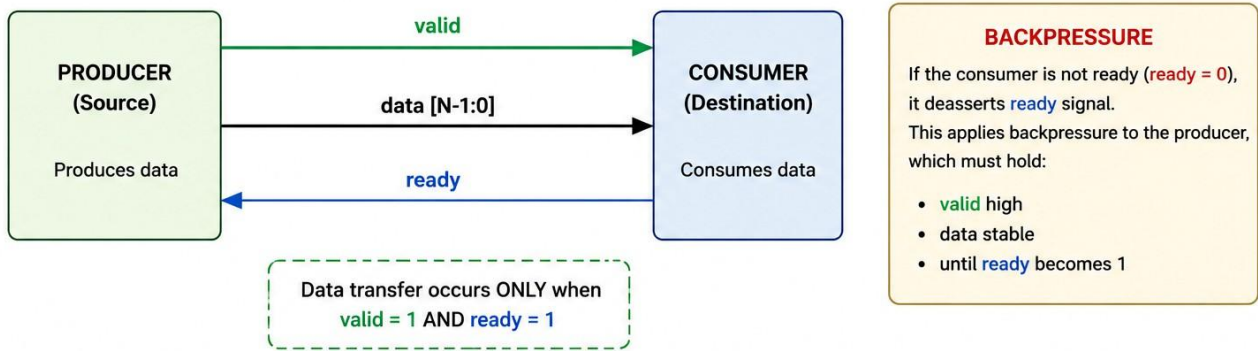
However, when I tried to feed multiple matrices continuously, I discovered a logical problem. There was no logic saying:

"Stop. I am still processing the current matrix. Don't send another one yet. There was also no mechanism to stop the entire MAC block when the output block was not ready to accept new data.

The problem is that **the data-processing latency is not always predictable**. It might take one cycle, or it could take thousands of cycles depending on the situation. Therefore, we need a mechanism that can tolerate variable latency and prevent data from being lost or overwritten. To solve this problem, I used the **valid-ready handshake protocol** in this design.

The **valid** signal indicates that the producer has valid data available, while the **ready** signal indicates that the consumer is able to accept it. Data is transferred only when both **valid** and **ready** are asserted. This simple mechanism allows different blocks to operate at different speeds while providing a clean way to apply **backpressure** when a downstream block is not ready.

VALID-READY HANDSHAKE



When **ready = 1**:

Data is accepted
(handshake occurs if **valid = 1**)



When **ready = 0**:

Data is stalled
(producer holds **valid** & data stable)



Backpressure prevents
overflow and data loss



Look at the `fetch_ready` and `pc_valid` signals. Both are high at this time, and the current PC is `0x000`.

Since there is a pipeline between **fetch** and **PC generation**, in the same cycle we send the request to the instruction memory (IMEM). The IMEM is ready to accept the request. A request transaction occurs when both `imem_req_valid` and `imem_req_ready` are high. If either of these signals is low, the transaction cannot occur, and we have to wait.

The fetch unit is now waiting for the instruction response. The `imem_resp_ready` signal is sent from the fetch unit to the memory, and it is high, meaning that the fetch unit is ready to receive the instruction.

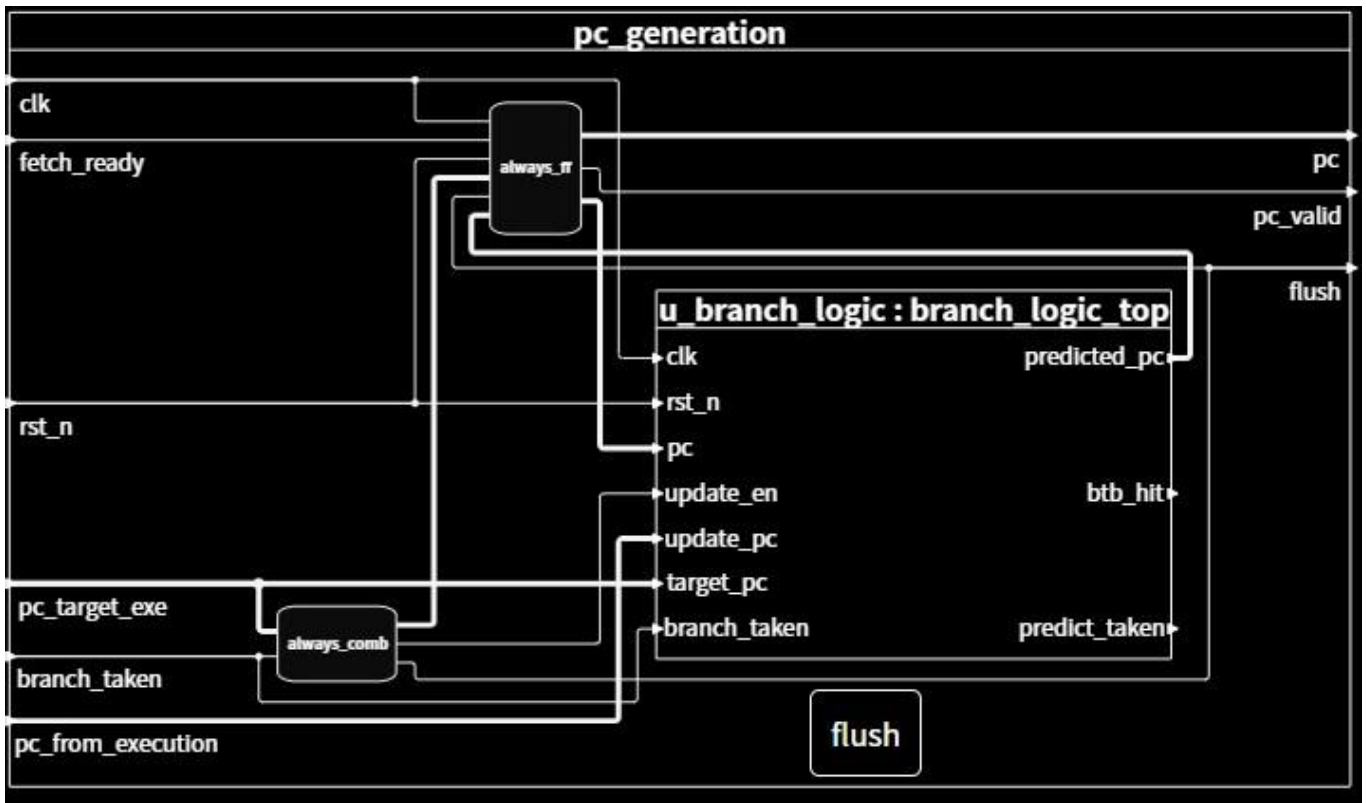
However, the actual response transaction occurs only when both `imem_resp_valid` and `imem_resp_ready` are high. You can see this transaction highlighted by the **orange line** in the waveform.

So, in both directions, the same valid-ready handshake is used:

- **Request:** imem_req_valid && imem_req_ready
- **Response:** imem_resp_valid && imem_resp_ready

This allows the fetch unit and instruction memory to operate independently and naturally provides **backpressure** whenever either side is not ready.

pc_generation block



Module pc_generation

This design unit is implemented in pc_generation.sv

Parameters and Ports

Parameters

Name	Type	Default Value	Description
XLEN	int	64	

Ports

Name	Direction	Type	Description
clk	input	wire logic	Clock and reset
rst_n	input	wire logic	
fetch_ready	input	wire logic	Handshake with fetch unit
pc_valid	output	logic	
pc_from_execution	input	wire logic [XLEN-1:0]	Correction inputs from pipeline stages
pc_target_exe	input	wire logic [XLEN-1:0]	Resolved/correct branch target
branch_taken	input	wire logic	EX: branch/jump was actually taken
pc	output	logic [XLEN-1:0]	PC output to fetch unit
flush	output	logic	Pipeline flush output (one-cycle pulse)

PC_GENERATION is one of the most important parts of a modern processor because it has a major impact on performance. To build a truly effective speculative machine, we need a very good **PC-generation system**.

In my machine, there is currently only **one stage between fetch and execution: the instruction decode stage**, which depends on the predicted PC. If the branch-prediction logic fails to predict the correct PC and an incorrect PC enters the decode stage, I simply assert the **flush signal** to clear the incorrect instruction. In this case, the penalty is only **one cycle**, which is not very significant.

Now consider a scenario where my PC-generation logic contains many modules. I need to read the **BTB**, compare the current PC with the BTB entries, determine whether there is a prediction, and then generate the predicted PC. This entire process takes time.

It is possible that the timing report shows that this logic violates the desired timing constraint. In that case, I may need to add another pipeline stage to the fetch stage and divide the PCgeneration logic into multiple stages.

However, this creates another problem. If the PC prediction is wrong, I may now need to **flush two instructions** instead of one.

This becomes even more important in a modern **P-core**. Many modern high-performance processors are superscalar and can potentially execute several instructions per cycle. For example, if a processor can process **8 instructions per cycle** and there are **4 pipeline stages before execution**, a wrong PC prediction could potentially cause up to:

$$4 \times 8 = 32$$

instructions to be in flight along the wrong path.

Therefore, PC generation creates an important trade-off:

More PC-generation logic → potentially better prediction → but longer critical path

More pipeline stages → higher clock frequency → but larger misprediction penalty

For a speculative processor, PC generation is therefore not just about calculating the next PC. It directly affects **clock frequency, branch-misprediction penalty, pipeline depth, instruction throughput, and ultimately overall performance.**

My design

What do we want: the **best solution or the optimal solution?**

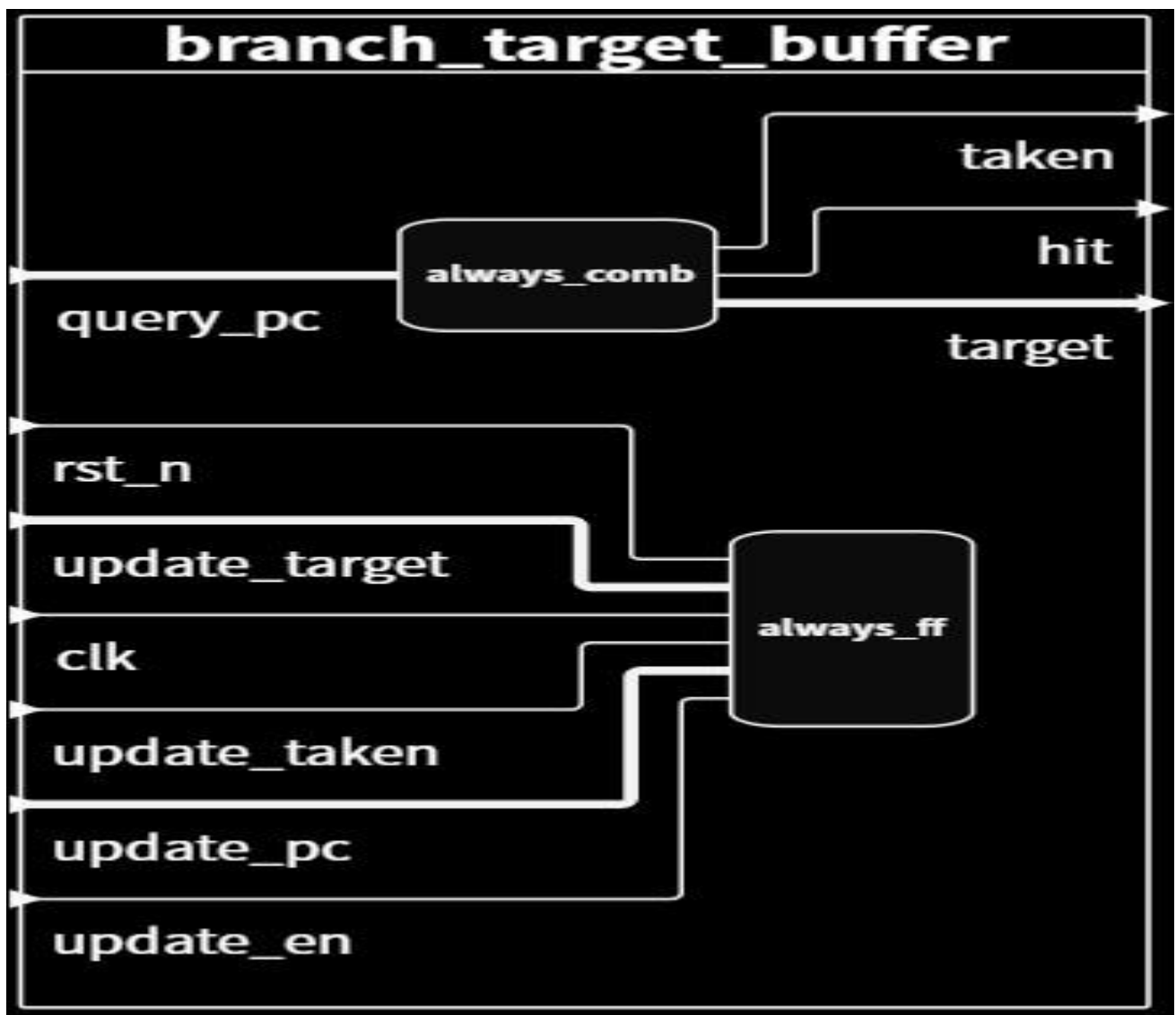
I know a little bit about **Apple Silicon**. They design highly optimized PCs and laptops, and everyone knows the outcome of that approach.

So, designing a **very powerful branch predictor** is not necessarily the best choice. Instead, what type of predictor do we actually want, and what are the penalties involved?

If the predictor fails, there is only a **one-cycle penalty**, because the pipeline flush takes just one cycle. In that case, making the predictor extremely sophisticated may not be worthwhile. A larger BTB, more complex prediction logic, or additional cycles to predict the next PC can actually add more penalty, as I discussed above.

As my design contains a **limited number of instructions**, I keep **precise exception handling** simple. My purpose is different; I am designing it with a specific goal in mind.

Since **PC generation and instruction fetch are in the same stage**, I need to make PC generation much faster. Therefore, I added a **BTB with only 16 entries**.



Module branch_target_buffer

This design unit is implemented in `branch_target_buffer.sv`

Parameters and Ports

Parameters

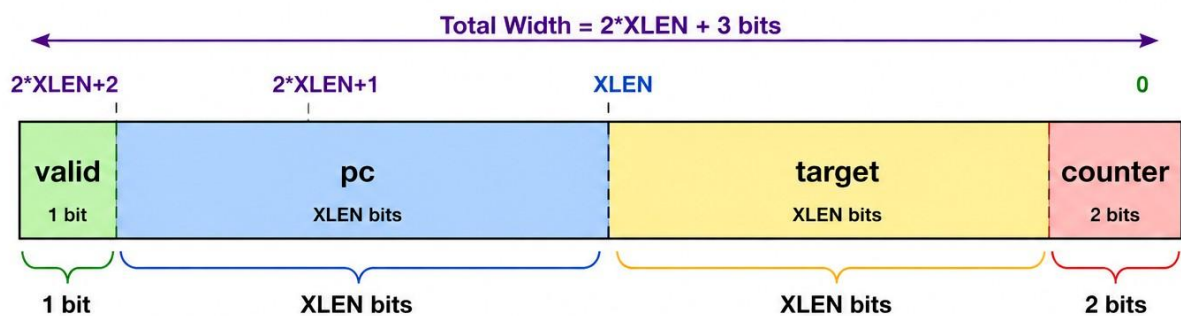
Name	Type	Default Value	Description
XLEN	int	64	
BTB_ENTRIES	int	16	

Ports

Name	Direction	Type	Description
clk	input	wire logic	
rst_n	input	wire logic	
update_en	input	wire logic	Update interface
update_pc	input	wire logic [XLEN-1:0]	
update_target	input	wire logic [XLEN-1:0]	
update_taken	input	wire logic	
query_pc	input	wire logic [XLEN-1:0]	Query interface
hit	output	logic	Outputs
target	output	logic [XLEN-1:0]	
taken	output	logic	

what are the entries in the buffer ?

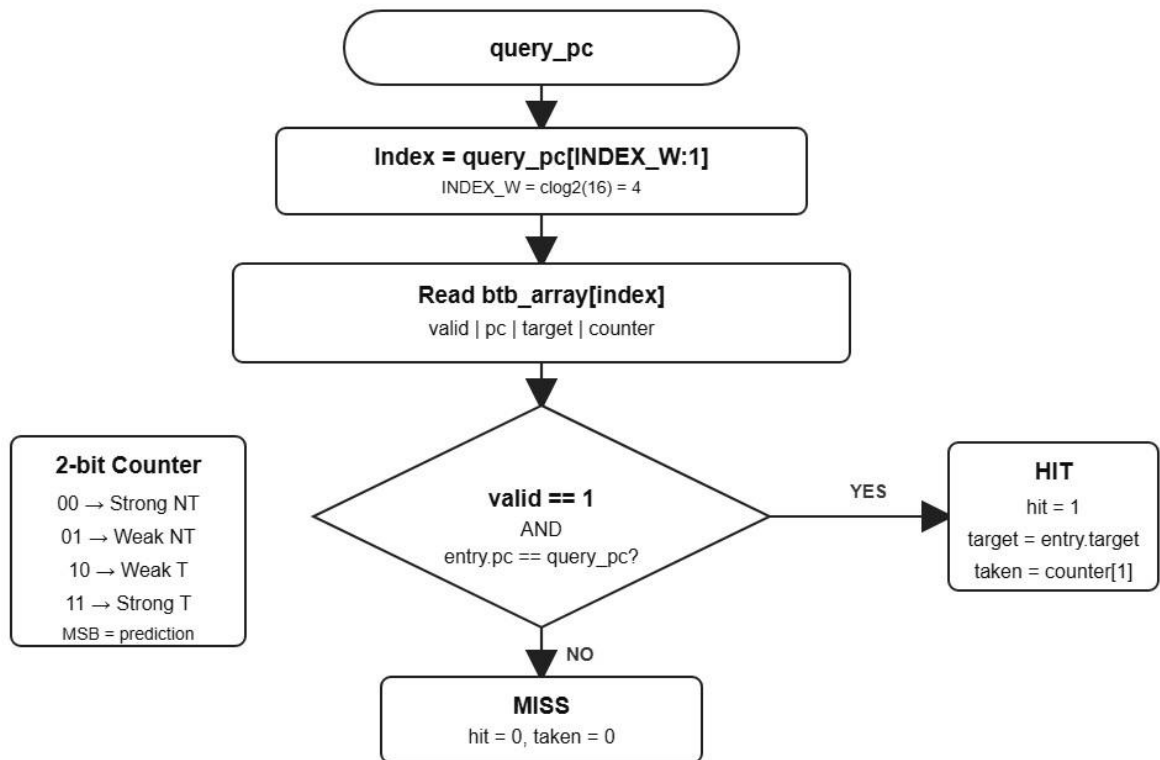
btb_entry_t (packed)



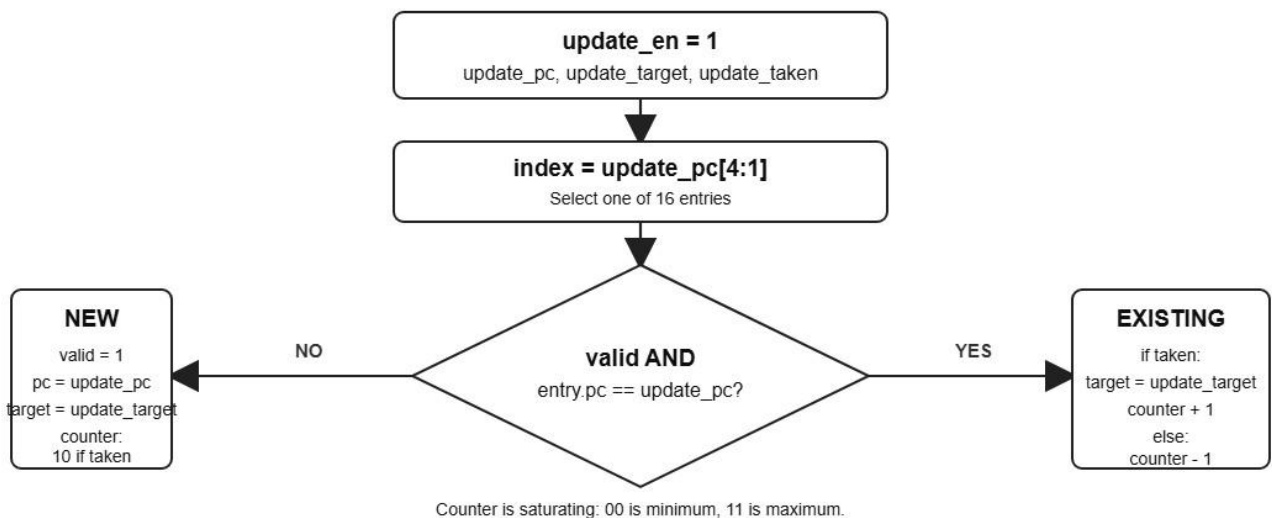
Field	Width	Description	Values / Notes
valid	1 bit	Indicates if the entry is valid	0 = invalid 1 = valid
pc	XLEN bits	Branch instruction PC	Stores the PC of the branch
target	XLEN bits	Predicted target address	Target address of the branch
counter	2 bits	2-bit saturating predictor	00 = Strongly Not Taken 01 = Weakly Not Taken 10 = Weakly Taken 11 = Strongly Taken

16-Entry BTB + 2-Bit Saturating Predictor

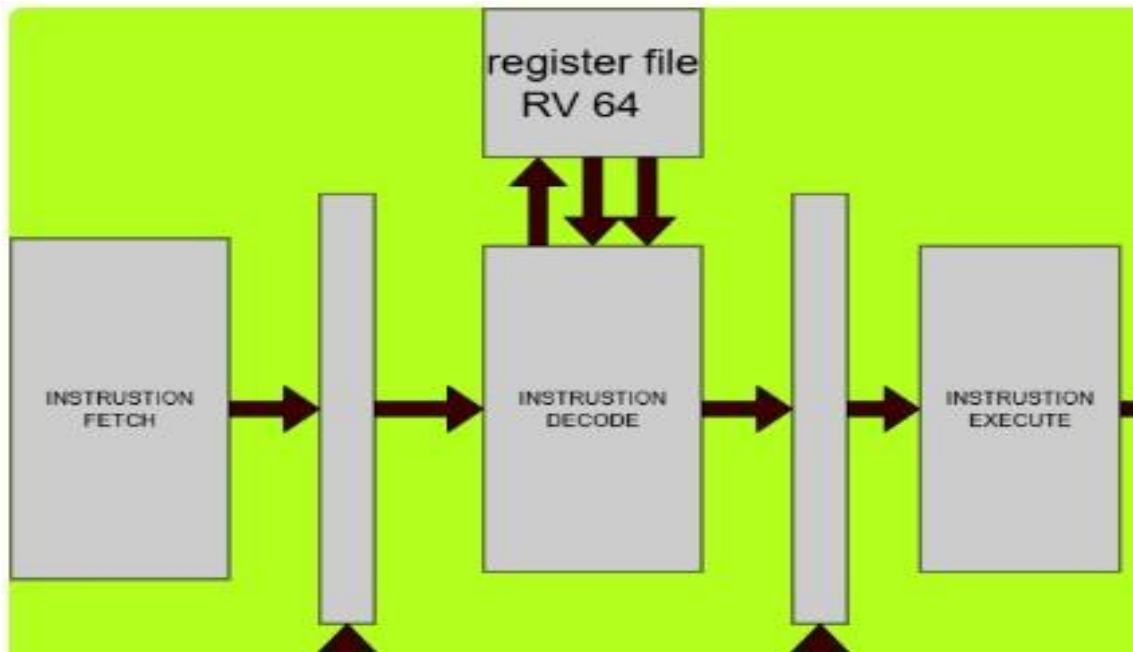
BTB QUERY / PREDICTION



BTB UPDATE / TRAINING

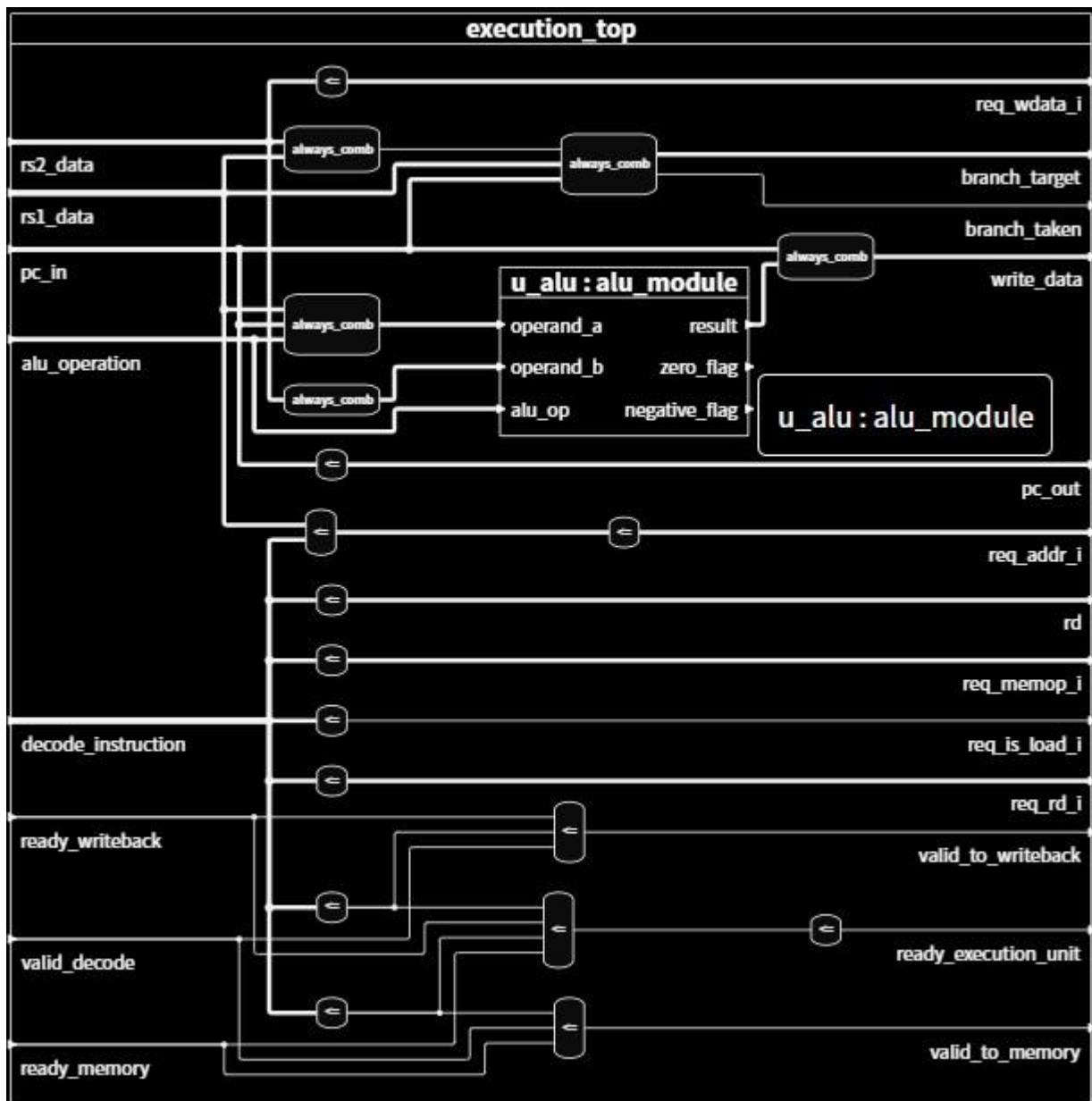


How do we check whether the prediction is correct or wrong?



Take an example: there is a **speculative PC** in the decode stage, and the execution unit executes the decoded instruction, which causes a branch. The execution unit then generates the **new PC**. If we compare the PC in the decode stage with the PC produced by the execution stage, we can easily determine whether the **PC prediction was correct or wrong**.

Execute Stage and ALU



Module execution_top

This design unit is implemented in `execution_top.sv`

Parameters and Ports

Parameters

Name	Type	Default Value	Description
XLEN	int	64	

Ports

Name	Direction	Type	Description
valid_decode	input	wire logic	Handshake from Decode stage
ready_execution_unit	output	logic	
rs1_data	input	wire logic [XLEN-1:0]	Operands forwarded from Decode / Register-File stage
rs2_data	input	wire logic [XLEN-1:0]	
pc_in	input	wire logic [XLEN-1:0]	Current PC (AUIPC, branch target, return address)
decode_instruction	input	wire decode_pkg::decoded_instr_t	Decoded instruction bundle + ALU opcode
alu_operation	input	wire alu_pkg::alu_op_t	
branch_taken	output	logic	Branch resolution → Fetch / Branch Control Unit
branch_target	output	logic [XLEN-1:0]	
pc_out	output	logic [XLEN -1 :0]	
valid_to_memory	output	logic	Handshake to Memory stage (loads & stores)
ready_memory	input	wire logic	
req_addr_i	output	logic [XLEN-1:0]	effective address (rs1 + imm)
req_wdata_i	output	logic [XLEN-1:0]	store write data (rs2)
req_memop_i	output	decode_pkg::mem_op_e	
req_rd_i	output	logic [4:0]	rd addr for load→WB path
req_is_load_i	output	logic	1 = this request is a load (for load-use hazard detection)
valid_to_writeback	output	logic	Handshake to Write-Back stage
ready_writeback	input	wire logic	
rd	output	logic [4:0]	
write_data	output	logic [XLEN-1:0]	ALU result or PC+4 (JAL/JALR)

Whatever information comes from the decode stage is represented in the form of a structure like this:

```
typedef struct packed {
```

```

    instr_type_e instr_type;    // [2:0]
    mem_op_e     mem_op;       // [3:0]
    branch_op_e  branch_op;    // [2:0]
    csr_op_e     csr_op;       // [2:0]
    sys_op_e     sys_op;       // [2:0]

    logic [4:0]  rs1;           // Register addresses
    logic [4:0]  rs2;
    logic [4:0]  rd;

    logic signed [63:0] imm;    // Immediate (sign-extended 64-bit)
    logic [11:0]  csr_addr;     // CSR address (12-bit)
    logic [4:0]   zimm;         // ZIMM field (5-bit unsigned, CSR immediate
variants)

    // Control flags
    logic rs1_used;
    logic rs2_used;
    logic rd_used;
    logic imm_used;
    logic mem_read;
    logic mem_write;
    logic branch;
    logic jump;
    logic csr_access;
    logic sys_call;

} decoded_instr_t;

```

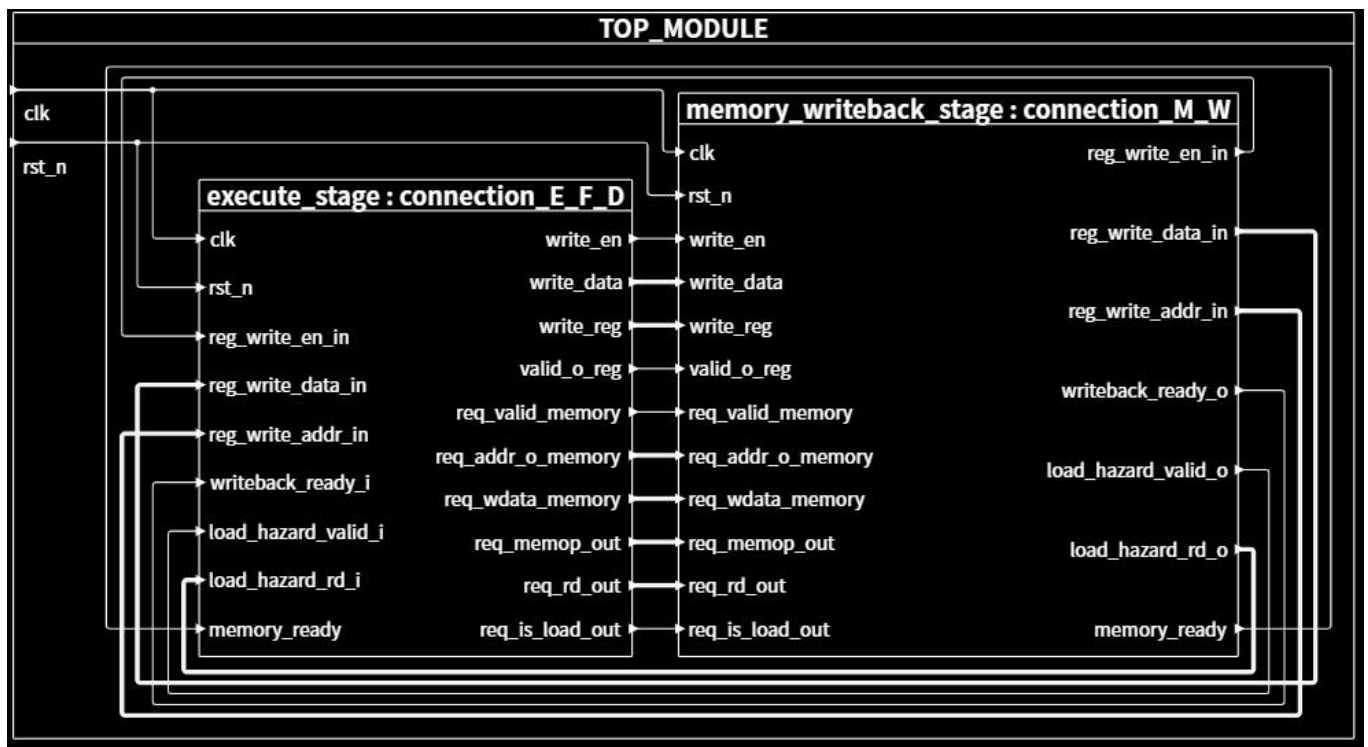
Using a structure provides several benefits. It makes the design easier to **handle, understand, and verify**, especially as the complexity of the processor increases. Instead of passing many individual control signals and fields between pipeline stages, the entire decoded instruction information can be passed as a single structured object.

This also improves **code readability, maintainability, and verification**, because related information is grouped together logically.

In the execution stage, there is an ALU module, and I am implementing all the instructions in it in a simple way, like $c = a + b$. I am also applying the same approach for the multiplier. I have worked on this project for a long time, and I have learned some things after I designed it. For example, using $c = a * b$ is not good practice because I am uncertain about the appropriate microarchitecture of the multiplier. A fine level of detail is required for the core, as everyone wants a high clock speed. Therefore, understanding the latency and how much it will create is essential.

Memory Access and Write-Back

I connected two module in top module like



The idea behind this is to simplify the verification process and to account for stalls and flushes, as branch prediction affects up to the execution stage, and stalls also impact data dependencies such as RAW, WAW, and WAR up to the execution stage.

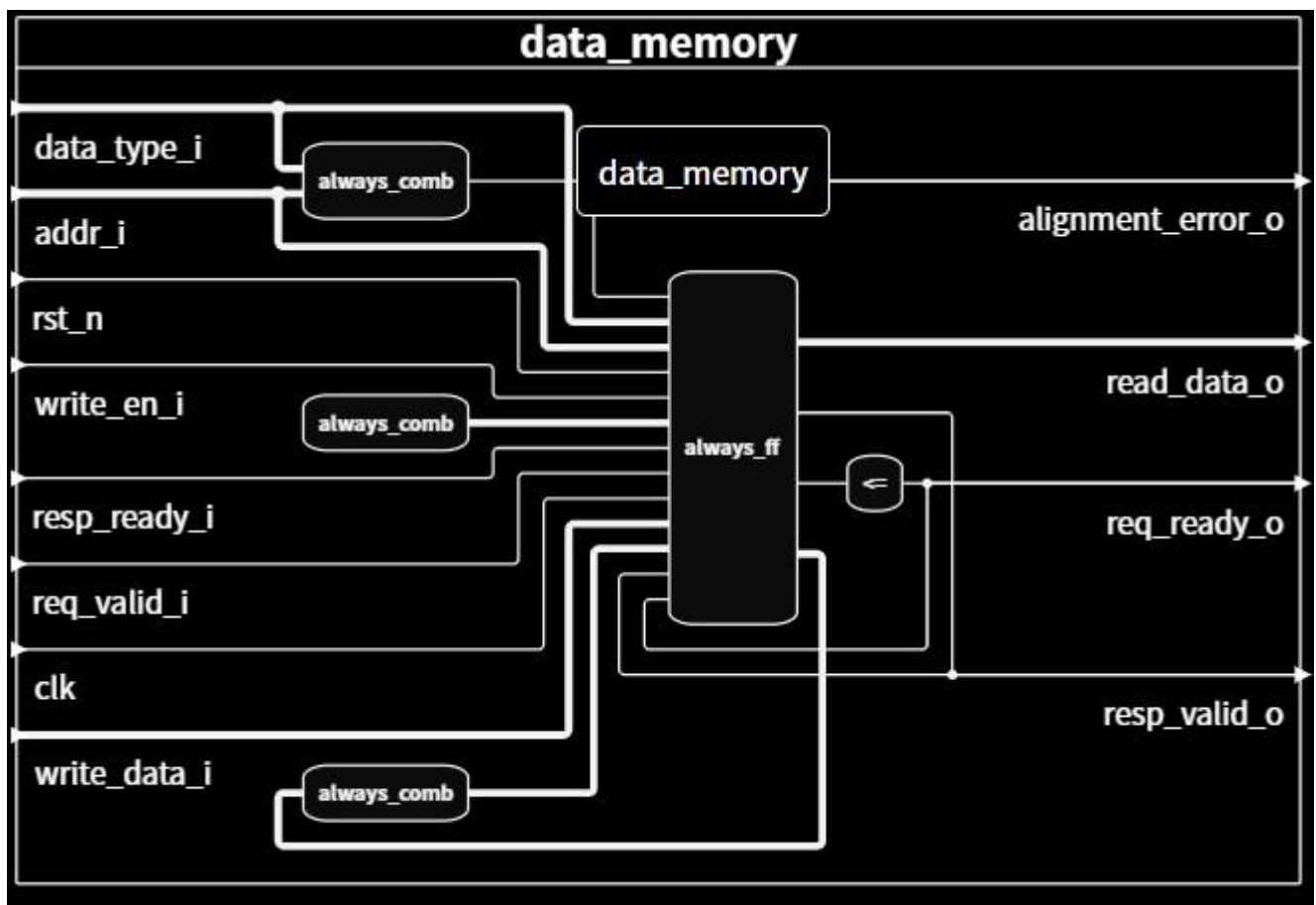
How Memory Access Occurs

The data memory is implemented as a separate module. I simulate the same latency for the data memory as I do for the instruction memory.

The execution stage sends a memory access request to the data memory. The memory then responds to the request. Before performing the access, the memory checks whether the address is properly aligned according to the data type being accessed.

If the address is not properly aligned, the memory generates a misaligned access error.

There is no dedicated Load/Store Unit because this machine is a non-speculative, in-order machine. It never executes predicted instructions.



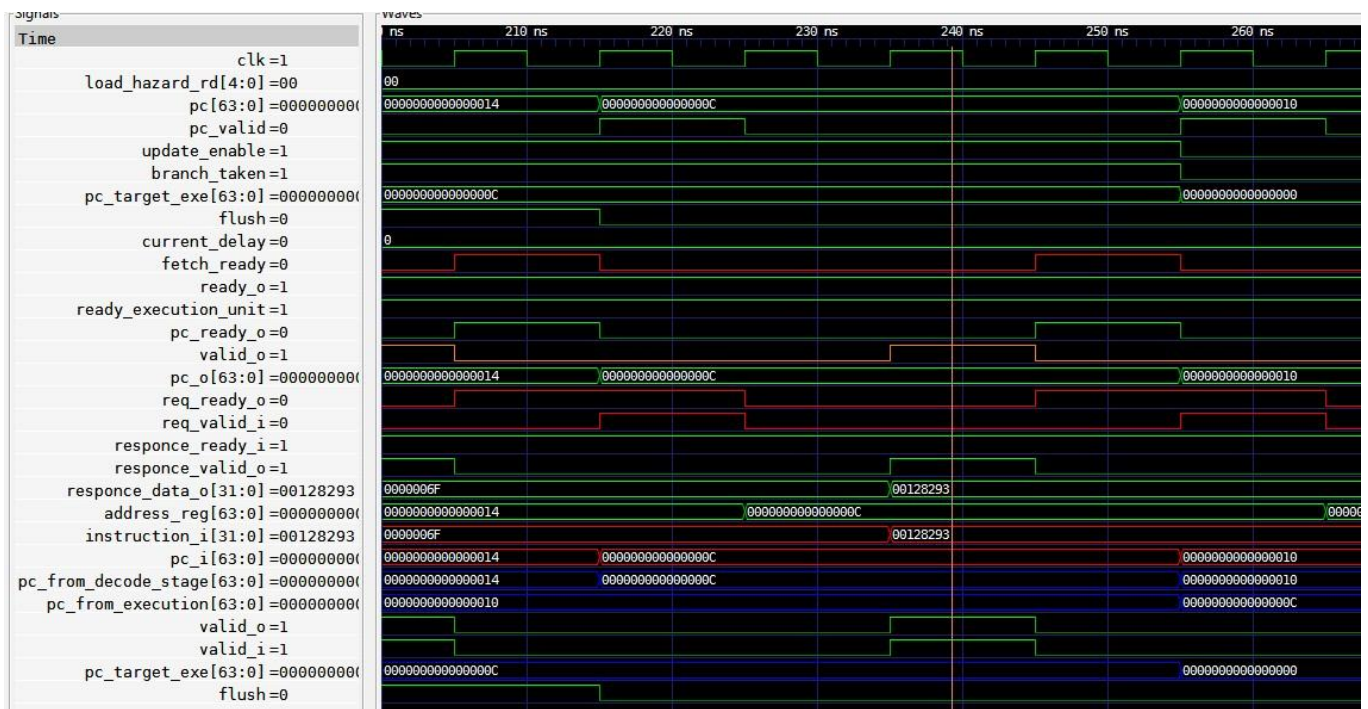
this is the main module of memory which is in memory file

Pipeline Registers

At the beginning of the project, I realized that dedicated pipeline stages were required, so I added them. Each module has a separate pipeline register/module, while the top-level module is primarily combinational.

For example, the Decode stage is purely combinational, so I needed to add a dedicated pipeline register after it to properly separate the stages.

However, this is not the case for the Fetch stage. If I add an additional pipeline register to the Fetch stage, it would introduce unnecessary latency into the pipeline. Therefore, I did not add a separate pipeline stage for Fetch.



The orange vertical line indicates that the data of the instruction is directly passed to the next stage. The data is captured if the validity is high.

The same applies to the Write-Back stage. I do not need an additional pipeline stage for the write operation because the register file itself uses synchronous writing. If I add another pipeline register, it would create two registers back-to-back, adding an extra stage without providing any real benefit.

Data Hazards and Hazard Handling

Hazards are handled in the Decode stage. My Decode stage is connected to the register file, and I provide the operands directly from the Decode stage.

Data hazards such as RAW (Read After Write) hazards are resolved because the result is written back to the register file after one cycle in my processor.

For most instructions, such as an ADD instruction, execution takes four pipeline stages:

Fetch → Decode → Execute → Write Back

For a load instruction, execution takes five pipeline stages:

Fetch → Decode → Execute → Memory → Write Back

In the first case, there is a one-cycle delay in my processor due to the memory latency. Because of this memory latency, the entire pipeline may stall for at least two cycles. This is a limitation of my current microarchitecture.

Therefore, there is no need to forward data from the Write-Back stage.

However, I have implemented dedicated logic to handle the load-use hazard:

```
logic load_use_hazard;

assign load_use_hazard =
    load_hazard_valid_i &&
    (load_hazard_rd_i != 5'd0) &&
    (
        (decoded_instr_reg.rs1_used && decoded_instr_reg.rs1 ==
load_hazard_rd_i) ||
        (decoded_instr_reg.rs2_used && decoded_instr_reg.rs2 ==
load_hazard_rd_i)
    );

assign ready_o = ready_i && !load_use_hazard;
```

This logic is implemented in the Decode pipeline stage. It is better to look at the Decode stage to understand how the hazard detection and stalling mechanism work.

For a load-use hazard, there is no fixed latency at which the required data will become available. Therefore, if the required data is not yet available, the processor must stall. In my implementation, this is done by preventing the instruction from advancing to the Execute stage until the required data becomes available.

Control Hazards and Branch Handling

I explained the branch predictor, prediction logic, and prediction algorithm above.

Here, I want to discuss the pipeline flush mechanism. When a branch misprediction occurs, I flush two pipeline stages: the Fetch stage and the Decode stage.

When the flush signal goes high, the corresponding pipeline registers are cleared and set to zero. The pipeline remains stalled/cleared until the flush condition is resolved, after which instruction fetching resumes from the correct target address.

Processor Control Unit

In many processor designs, there is a dedicated control unit implemented as an FSM, and the FSM controls the different modules and pipeline stages.

In my processor, there is no dedicated centralized control unit. Instead, each pipeline stage is designed to be self-contained and communicates with the other stages using a defined protocol.

The control signals required for execution are generated by the Execute stage, while the control information required for Write-Back is also generated and passed forward by the Execute stage.

For higher-order control, the pipeline control logic is distributed across the different modules, with the overall pipeline coordination handled at the top level.

Verification and Testing

When I started implementing the microarchitecture, I decided that each pipeline stage should have its own testbench because verification is just as important as the design itself.

Initially, I verified each stage independently using separate testbenches. However, I encountered problems when connecting the individual stages together—the complete pipeline did not always behave as I expected.

Therefore, I adopted another approach. I connected all the modules together and verified the complete processor using a top-level testbench. The testbench generates the clock and feeds instructions from a hex file into the instruction memory.

From there, I trace the signals backward to identify the root cause of any problem—for example, why a `valid` signal is not becoming high or why an instruction is not being fetched correctly. This approach can be time-consuming, but I find it very useful for debugging because it allows me to observe the actual behavior of the complete pipeline.

It is also very easy to create corner-case tests using assembly instructions. For example, to verify whether the stall mechanism is working correctly, I can use a simple loop such as:

```
addi x5, x0, 0      # x5 = 0
addi x6, x0, 10     # x6 = 10
```

```
loop:
addi x5, x5, 1      # x5++
blt  x5, x6, loop

jal  x0, 0          #infinite loop
```

I can assemble this code, generate the corresponding hex file, and execute it on the processor. This allows me to test the pipeline behavior without having to write additional SystemVerilog code for every test case.

For Verilator-based simulation, this approach also significantly reduces the development and testing time because I can generate new test programs and run them without modifying the SystemVerilog testbench for each individual case.

I also developed an assembler and compiler flow to make this process easier.

Note: The compiler and assembler implementations were not written by me.

For reference and verification, I also implemented a golden model in C++. The golden model was developed with the help of Claude. Simulating the pipeline behavior in C++ is somewhat more difficult than implementing the RTL, but the model is working and provides a useful reference for comparing the RTL results.

My primary focus in this project is the **RTL design and microarchitecture**. The assembler, compiler flow, test infrastructure, and golden model are mainly automation and verification tools that support the main work.

Limitations and Future Improvements

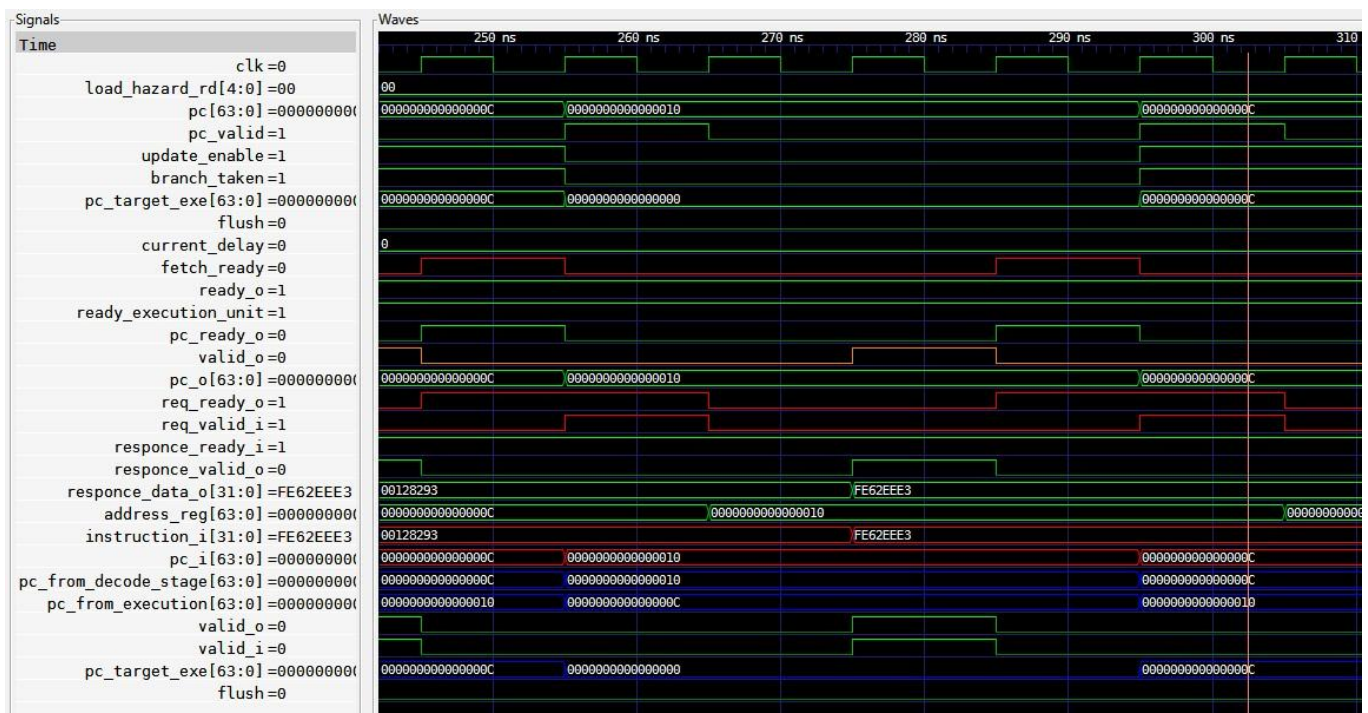
The way I explain the features of my processor, I also want to explain its limitations and design challenges. These limitations are not RTL bugs; they are inherent trade-offs in my microarchitecture.

Valid-Ready Protocol

My processor uses the Valid-Ready protocol extensively. It is a very robust protocol and works well for controlling data flow between pipeline stages. However, it can become a limitation when fine-grained control of latency and throughput is required.

In my design, the Valid-Ready protocol can introduce additional latency that is not always necessary. These extra cycles can affect the overall pipeline throughput, particularly in cases where the latency between stages needs to be tightly controlled.

Therefore, although the Valid-Ready protocol provides a clean and robust way to handle backpressure and pipeline control, it also introduces some performance overhead in my microarchitecture. This is a design trade-off rather than an RTL bug.



Ideally, the pipeline should behave like this: in the first cycle, the PC is `0x10`, and in the next cycle, the PC should advance to `0x1C`. However, this does not happen in my current implementation because of the Valid-Ready protocol. The protocol introduces two additional cycles of latency, and the instruction memory itself adds another two cycles.

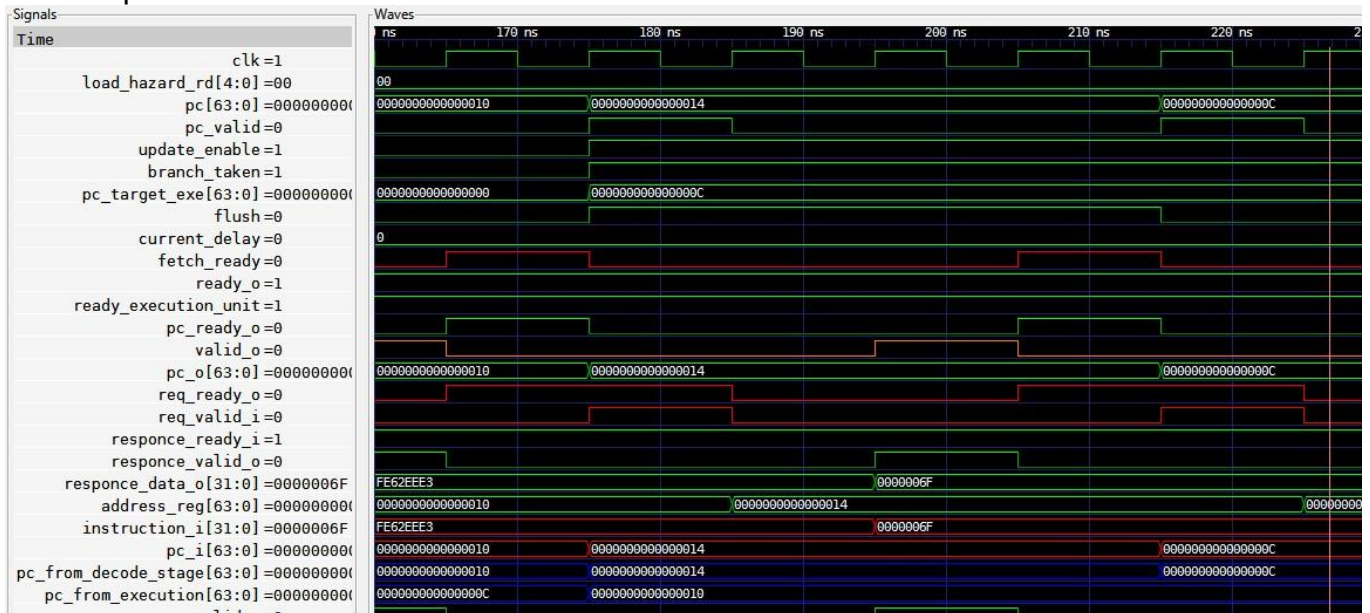
The backpressure mechanism I implemented is therefore not the optimal solution for achieving the best pipeline performance. A more efficient pipeline-control protocol could reduce this additional latency.

However, changing the protocol at this stage would require significant changes to the RTL architecture and would effectively mean redesigning a large portion of the pipeline. It would take considerable additional development time.

The important point is that the pipeline is fully implemented and functional. The current design achieves the intended functionality, but the performance benefit is not as high as it could be because of the additional latency introduced by the pipeline-control protocol.

This is therefore a **microarchitectural limitation and design trade-off**, not an RTL correctness bug.

another problem is with flush



When a PC value such as 0x14 is predicted and the prediction turns out to be incorrect, a pipeline flush is triggered. However, the processor may still fetch the instruction corresponding to the incorrect PC before the flush takes effect. This additional fetch does not reduce the pipeline stall and therefore adds unnecessary latency.

Ideally, if the current cycle has PC = 0x14 and the next cycle should redirect to PC = 0x10, I would want to prevent the instruction at PC = 0x14 from being fetched once the misprediction is detected.

However, due to the current pipeline-control and Valid-Ready protocol, the incorrect instruction can still be fetched. This introduces an additional cycle of latency during branch recovery.

This is another limitation of the current microarchitecture. The pipeline is functionally correct—the incorrect instruction is eventually flushed—but the control protocol prevents the design from recovering from a branch misprediction as efficiently as it could.

problem with the latency

```
always_comb begin
    redirect_pc = 0 ; //default value;
    flush = 0 ;
```

```

branch_correct = 0 ;
update_enable = 0 ;
if(branch_taken) begin
    if( pc_from_decode_stage == pc_target_exe )begin
        update_enable = branch_taken;
        branch_correct = 1'b1 ;
    end else begin
        redirect_pc    = pc_target_exe;
        flush          = branch_taken;
        update_enable = branch_taken;
        branch_correct = 1'b0 ;
    end
end
end
end

```

The branch decision signal is generated in the Execute stage after some combinational processing. The decision is made based on the branch condition, such as whether the branch is taken. Because this signal is generated after the required processing, it can introduce additional latency, which can be problematic for the overall pipeline timing.

One possible improvement would be to introduce a separate **PC Generation (PG) stage**. This could allow the branch-related combinational logic to be divided into smaller portions and reduce the amount of combinational logic in a single pipeline stage.

By splitting the larger combinational logic across multiple stages, we could potentially improve the timing and reduce the latency associated with generating the branch decision and target PC.

Conclusion

This project has been a valuable learning experience in processor design and microarchitecture. I was able to solve several of the challenges I encountered during the implementation, while some of the limitations and problems were introduced by my own architectural decisions.

The design may not be perfect, and there are certainly areas where the microarchitecture could be improved, particularly in terms of pipeline latency, throughput, and control mechanisms. However, the goal of the project was not simply to create a perfect processor—it was to understand how a processor pipeline actually works and to gain practical experience implementing one.

Through this project, I gained a much deeper understanding of the real-world challenges involved in pipeline design, including hazards, stalls, backpressure, branch mispredictions, pipeline flushing, memory latency, and stage-to-stage communication.

More importantly, I developed practical RTL debugging skills. When the complete pipeline did not behave as expected, I had to trace signals across multiple stages, identify where the control or data flow was breaking, and understand why the pipeline was behaving differently from what I expected.

Overall, this project gave me practical experience that goes beyond theoretical knowledge. I learned not only how to design a processor pipeline, but also how to debug it, identify architectural limitations, and make engineering trade-offs when there is no perfect solution.

Some problems I solved, some problems I created—but every problem taught me something new.

References

The following resources were used as references during the design and implementation of this processor project.

1. SAFARI Research Group — Computer Architecture

Computer architecture lectures and research material from the SAFARI Research Group were used as references for processor architecture, memory systems, caches, pipelining, and modern computer architecture concepts.

- **Website:** <https://safari.ethz.ch/>

2. Frank Kagan Gürkaynak — Digital Design, ETH Zürich

Digital design lectures and course material were used as references for RTL design, digital logic, pipelining, and hardware implementation concepts.

- **Institution:** ETH Zürich

3. Prof. Adam Teman — VLSI Design

VLSI design lectures and course material were used as references for VLSI implementation, timing, digital circuit design, and hardware design methodology.

- **Instructor:** Prof. Adam Teman
EnICS Labs

4. MIT OpenCourseWare — Complex Digital Systems

MIT OpenCourseWare's **6.884 — Complex Digital Systems** was used as a reference for complex digital-system design, Verilog, synthesis, CMOS technology, architectural exploration, and hardware implementation.

- **Course:** 6.884 — Complex Digital Systems
- **Link:** <https://ocw.mit.edu/courses/6-884-complex-digital-systems-spring-2005/>

5. MIT OpenCourseWare — Computation Structures

MIT OpenCourseWare's **6.004 — Computation Structures** was used as a reference for digitalsystem architecture, processor design, pipelining, performance, and hardware debugging.

- **Course:** 6.004 — Computation Structures
- **Link:** <https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/>

6. Computer Architecture: A Quantitative Approach — 6th Edition

John L. Hennessy and David A. Patterson

Computer Architecture: A Quantitative Approach, 6th Edition, Morgan Kaufmann, 2017.

This book was used as a major reference for computer architecture, processor design, pipelining, performance analysis, memory systems, and architectural trade-offs.

- **Publisher:** Morgan Kaufmann / Elsevier
- **Edition:** 6th Edition
- **Year:** 2017
- **Link:** <https://shop.elsevier.com/books/computer-architecture/hennessy/978-0-12-811905-1>

Note

These resources were used for **learning, reference, and understanding architectural concepts**. The processor RTL and microarchitectural implementation in this project were developed as part of the project itself.